

Annotation Guidelines

How the annotator applies `taxonomy-v0.md` to a normalized trajectory. The annotator is the **LLM judge** (validated in RQ2) — there is no human annotation in this project. This document is used verbatim as the judge's instructions, so every rule must be explicit and mechanical — no unstated conventions.

What you annotate

One **annotation** = one error occurrence:

Field	Value
<code>trajectory_id</code>	The trajectory being annotated
<code>event_span</code>	<code>[start, end]</code> event indices where the error <i>manifests</i> (usually a single event)
<code>cognitive_function</code>	<code>memory</code> / <code>reflection</code> / <code>planning</code> / <code>action</code> / <code>system</code>
<code>error_type</code>	Taxonomy code (e.g., <code>RFL-1</code>) or <code>NEW-?</code>
<code>severity</code>	<code>low</code> / <code>medium</code> / <code>high</code> (see scale below)
<code>root_cause</code>	<code>true</code> for the earliest correctable error (see heuristic)
<code>cascade_of</code>	Annotation id of the root cause this error propagated from, if any
<code>rationale</code>	1–3 sentences quoting the evidence in the trajectory
<code>confidence</code>	<code>certain</code> / <code>probable</code> / <code>speculative</code>
<code>proposed_category</code>	Free text, only with <code>NEW-?</code>

A failed trajectory typically gets several annotations (TRAIL average: ~5.7). A *successful* trajectory can also carry annotations (errors the agent recovered from) — annotate those too; they matter for the stochastic-vs-systematic analysis (RQ3).

Procedure

- Read the task first.** Read the instruction and, if available, the oracle/test criteria before reading any agent behavior, and note the explicit constraints.
- First pass — comprehension.** Read the whole trajectory without labeling. Note the agent's apparent plan and where the outcome diverged from the goal.
- Second pass — mark candidate errors.** Walk events in order; mark every event where something is objectively wrong per a taxonomy definition. Quote the evidence.
- Classify each candidate.** Choose cognitive function first ("what faculty failed?"), then the error type within it. Apply the decision rules in the taxonomy; use the tie-breakers below if two labels still fit.
- Identify the root cause.** Apply the root-cause heuristic; set `root_cause: true` on exactly one annotation per independent failure chain (a trajectory can rarely have two independent chains). Link downstream consequences via `cascade_of`.

6. **Severity pass.** Assign severity per the scale.
7. **Completeness check.** If the task failed, at least one annotation must exist whose severity is **high** or whose chain explains the failure. If you cannot produce one, either the failure is SYS-3 (environment defect — needs evidence) or the taxonomy has a gap → use **NEW-?**.

Root-cause heuristic

The root cause is the **earliest** annotated error such that, had it been corrected at that point (counterfactually, with everything before it unchanged), the task would plausibly have succeeded within the remaining budget.

- Work backwards from the failure, then forwards from the start; they should meet at the same event.
- Mechanical terminal events are usually symptoms: a failing command (ACT-x) whose intention was already doomed points back to the PLN/MEM/RFL error that formed the intention.
- SYS-1 (budget exhaustion) is almost never a root cause — ask what consumed the budget.
- If two candidate root causes are both plausible, prefer the earlier one and set **confidence: probable** on both.

Tie-breakers for common confusions

Confusion	Rule
MEM-1 vs RFL-1	Was the information in an <i>earlier</i> event (memory) or in the output the agent is reacting to <i>right now</i> (reflection)?
MEM-2 vs RFL-5	False claim about what was <i>observed</i> (MEM-2) vs. about a <i>check the agent performed</i> (RFL-5).
MEM-3 vs PLN-1/PLN-2	Did the agent ever behave consistently with the requirement? Yes → MEM-3 (drift). No → PLN-1 if misread, PLN-2 if ignored.
RFL-2 vs RFL-3	Did the agent <i>judge</i> progress incorrectly (RFL-2) or simply never run an available check (RFL-3)? RFL-3 requires naming the concrete check that was skipped.
ACT-2 vs PLN-5	Wrong instrument for a feasible step (ACT-2) vs. a step no instrument could perform in this environment (PLN-5).
ACT-1/3/5 among themselves	Shell-syntax malformed → ACT-1. Command runs but wrong args → ACT-3. Produced <i>artifact</i> malformed → ACT-5.
PLN-3 vs RFL-4	The repetition itself is PLN-3; a wrong <i>diagnosis</i> driving varied-but-misdirected fixes is RFL-4. Both can co-occur (RFL-4 as root cause, PLN-3 as cascade).
Any agent error vs SYS-3	Before labeling SYS-3, state what evidence shows a correct agent would also have failed (e.g., missing dependency with no offline alternative).

Severity scale (adapted from TRAIL)

- **high** — this error, alone, is sufficient to fail the task (or did).
- **medium** — materially reduced success probability or wasted a significant fraction (>~20%) of the budget, but was survivable.

- **low** — local mistake, quickly recovered, negligible effect on the outcome.

Location conventions

- Annotate the error where it **manifests** — the first event containing direct evidence, not where its consequences appear.
- Omission errors (RFL-3) are located at the last event where performing the check was still natural — typically the event before task completion/submission.
- Spans longer than one event are for genuinely distributed errors (e.g., a PLN-3 loop spans all its iterations).

Hygiene

- Annotate only from the trajectory and task materials. Do not consult the agent's model identity, other trajectories of the same task, or leaderboard context while labeling (bias).
- One judge pass per trajectory; judge validity comes solely from the TRAIL agreement study (RQ2).
- Rationales must quote or index concrete evidence — a rationale that can't cite an event is **confidence: speculative**.